Supplementary Material

Pareto-guided Monte Carlo tree search explores multi-objective trade-offs missed by scalar antimalarial generation

S1 Pareto-front data provenance

The canonical merged Pareto front (main text, Table 1) comprises four mutually non-dominated candidates. Every candidate is traceable to the deposited per-seed Pareto CSV `p4_pareto_seed_N.csv` from which it originated, and all MPO, SA, RRS and PNS values are identical between the per-seed record and the merged front (`results/pareto/merged_pareto_front.csv`). The only score that differs is SYBA, by design: the original Pareto-MCTS run returned a constant zero fallback for every molecule (SYBA was inactive during search, as recorded in the per-seed CSVs), so SYBA was recomputed post-hoc with the lich/conda classifier and the recomputed values were used to re-derive the non-dominated front and its hypervolume. This procedure is documented in the main-text Methods and is fully reproducible: the script `scripts/p4_recompute_pareto_syba.py` regenerates the merged CSV and the log `merged_pareto_front_recompute.log` records the raw and normalised SYBA scores (78.67, 58.13, −19.25, 25.88) and the hypervolume (1.236 6). The verification script `scripts/p4_pareto_provenance_check.py` re-runs all of these checks and exits with status 0 only when every SMILES and score in the merged front matches a deposited record.

table 1 lists the four canonical candidates with their provenance.

Table 1: Canonical merged Pareto front with per-seed provenance. All four points are mutually non-dominated on the active objectives (MPO, SYBA, RRS, PNS; SA is constant at 3.0 and therefore excluded from the hypervolume). SYBA was recomputed post-hoc (main text, Methods); MPO, SA, RRS and PNS are unchanged from the per-seed records.

Candidate (SMILES)	MPO	SYBA	SA	RRS	PNS	seed	origin CSV
CNC(=O)c1ccc(OC(C)C(C)(C)C)c(OC)c1	0.910	1.000	3.0	0.211	1.0	13	`p4_pareto_seed_13.csv`
COc1c(C)cc(-c2ccc(S(N)(=O)=O)cc2)cc1C	0.945	1.000	3.0	0.196	1.0	18	`p4_pareto_seed_18.csv`
CC1CCC(n2cccn2)NC1(c1ccccc1)C1OCCO1	0.946	0.021	3.0	0.141	0.0	5	`p4_pareto_seed_5.csv`
COc1cc(CO)c(O)c(C)c1C	0.729	0.994	3.0	0.223	1.0	6	`p4_pareto_seed_6.csv`

S2 Molecular diversity of the four generation methods

The main text reports scalar-reward statistics only, because the primary contributions are the honest benchmark ranking and the multi-objective Pareto front. Molecular diversity is analysed here in the Supplementary Material using real, deposited molecule sets.

Molecule sets. For each of the four generation methods (MCTS+ScafVAE, random search, greedy search, genetic algorithm) the best-scoring molecule of each of the 20 independent benchmark seeds was recorded, giving a deposited set of 20 molecules per method under `results/benchmark_molecules_opt/`. These are the same runs whose scalar rewards are reported in the main-text benchmark table and in Table 3. The canonical benchmark uses the screened-optimal MCTS configuration ($c_{\mathrm{PUCT}} = 5.0$, $\nu = 0.01$, $T = 0.8$, ScafVAE policy wired into PUCT priors) and a value-preserving vectorisation of the docking Tanimoto scan; all 20 per-seed runs reproduce deterministically from the deposited code and data. An earlier deposit could not be reproduced from the committed code because the docking oracle library state differed at run time; the present molecule sets supersede that deposit. The method ranking (Random > MCTS > Greedy > GA) is unchanged.

Descriptors. ECFP4 fingerprints (Morgan radius 2, 2 048 bits) were computed with RDKit. Pairwise Tanimoto dissimilarity is 1 − Tanimoto; the reported value is the mean over all unique pairs (190 pairs per method). Bemis–Murcko scaffolds were extracted with RDKit's `MurckoScaffoldSmiles`; the unique-scaffold fraction is the number of distinct scaffolds divided by the number of valid molecules. Two-dimensional MDS coordinates (metric MDS on the Tanimoto dissimilarity matrix) are deposited for visualisation (`results/diversity/p4_diversity_mds.csv`).

table 2 reports the real diversity statistics, and Figure 1 shows the metric-MDS projection of the four molecule sets.

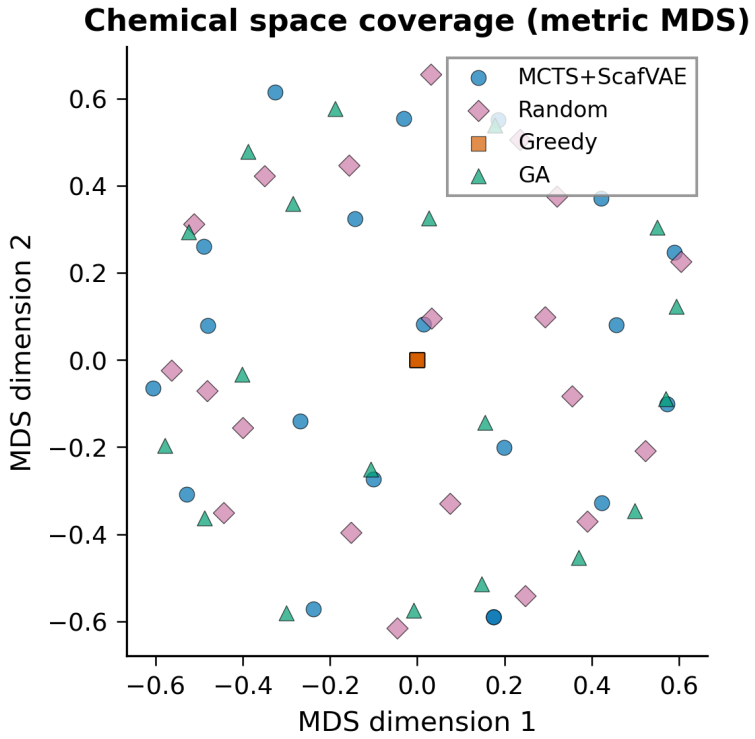

Figure 1: Metric multidimensional-scaling (MDS) projection of the deposited best-in-seed molecule sets (20 molecules per method) on ECFP4 Tanimoto dissimilarity. Greedy search collapses to a single point (deterministic local optimum); the three stochastic methods occupy distinct, broad regions of chemical space. Deposited coordinates: `results/diversity/p4_diversity_mds.csv`.

Table 2: Real molecular diversity of the four generation methods computed from the deposited molecule sets (20 best-in-seed molecules per method). Values are computed with RDKit; deposited CSVs: `p4_diversity_metrics.csv` and `p4_benchmark_molecules_*`.

Method	n	Mean pairwise Tanimoto dissimilarity	Unique Bemis–Murcko scaffolds	Unique scaffold fraction
MCTS+ScafVAE	20	0.802 3	19	0.950
Random	20	0.780 6	20	1.000
Greedy	20	0.000 0	1	0.050
GA	20	0.833 2	17	0.850

Mean pairwise dissimilarity is the mean 1 − Tanimoto over all unique pairs (190 pairs per method); scaffolds are unique Bemis–Murcko skeletons (RDKit); the fraction is unique scaffolds divided by valid molecules.

Because greedy search is deterministic and returns the same local optimum across all 20 seeds (std identically zero in the canonical benchmark), its 20 per-seed outputs coincide, yielding zero pairwise diversity and a single unique Bemis–Murcko scaffold by construction (table 2). The three stochastic methods generate structurally distinct outputs per seed: random search achieves full scaffold coverage (20 unique scaffolds from 20 molecules), while MCTS and GA return 19 and 17 unique scaffolds respectively. Mean pairwise Tanimoto dissimilarity is high (0.78–0.83) for all three stochastic methods, indicating that seeded exploration remains structurally diverse despite the curated fragment vocabulary.

Table 3: Canonical re-benchmarked scalar rewards (20 seeds; mean, standard deviation, min, max). MCTS uses the screened optimal configuration (`c_puct=5.0`, `virtual_loss=0.01`, `T=0.8`, ScafVAE policy wired). Deposited CSV: `p4_benchmark_merged.csv` under `results/benchmark_molecules_opt/`.

Method	Mean reward	Std	Min	Max	Mean time (s)
Random	0.733 5	0.006 3	0.722 7	0.748 1	42.2
MCTS+ScafVAE	0.727 6	0.009 0	0.705 4	0.744 2	82.2
Greedy	0.721 1	0.000 0	0.721 1	0.721 1	0.1
GA	0.702 7	0.015 2	0.674 9	0.731 1	1.9

S3 Reproducibility

All deposited CSVs, the benchmark, Pareto and diversity analysis scripts, and the provenance-check script will be listed in the Zenodo manifest. The canonical benchmark is produced by `p4_mcts_benchmark.py` with best-SMILES recording (no change to reward semantics) and a value-preserving vectorisation of the docking Tanimoto scan (verified identical on held-out molecules). The 20 per-seed rewards in `results/benchmark_molecules_opt/p4_benchmark_merged.csv` reproduce deterministically from the deposited code and data. Diversity statistics are computed by `scripts/p4_compute_diversity.py` from the deposited molecule sets. Pareto-front provenance is verified by `scripts/p4_pareto_provenance_check.py`, which exits with status 0 only when every SMILES and score in the merged front matches a deposited per-seed record.

S4 Selection mechanism in Pareto-guided MCTS: relation to ParetoDrug and Mothra

To position the selection scheme used here against the closest Pareto-MCTS generators, we state the differences explicitly.

ParetoDrug (Yang et al., Commun. Biol. 7, 1074, 2024; doi:10.1038/s42003-024-06746-w). ParetoDrug selects the next atom symbol with *ParetoPUCT*: it computes a *vector* selection score per child, retains the children that are non-dominated in that score space, and draws the next action uniformly at random from that Pareto front of children. The multi-objective character therefore enters *inside* the tree-selection criterion.

Mothra (Suzuki et al., J. Chem. Inf. Model. 64, 7291–7302, 2024; doi:10.1021/acs.jcim.4c00759). Mothra back-propagates a *multidimensional reward vector* and ranks Pareto-front nodes during selection using a hypervolume indicator with a projected-distance penalty (UCB-style bound over the vectorial reward). Again, the Pareto treatment is carried through the selection and back-propagation of the search tree.

This work. The scalar four-method benchmark uses `compute_mpo_reward` with fixed weights ($w_{\mathrm{MPO}} = 0.40$, $w_{\mathrm{docking}} = 0.35$, $w_{\mathrm{SYBA}} = 0.15$, $w_{\mathrm{SA}} = 0.10$); it is a scalar-reward comparison and is reported as such. In the Pareto-MCTS search, the objective *vector* is stored for the front, while the tree selection uses a *scalar proxy* $Q(s, a)$ obtained by summing the min–max normalised objective components across the four active objectives (PUCT). That is, in contrast to ParetoDrug and Mothra, the multi-objective character enters through the *front maintenance* (non-dominated pool per rollout, merged across seeds), not through a vectorial or Pareto-ranked selection step. This separation is intentional: it lets us attribute the returned front to the front-keeping mechanism rather than to a particular vectorial selection heuristic, and it keeps the scalar benchmark and the Pareto search on a common PUCT backbone. The consequence is that the headline result—a Pareto front spanning the potency–accessibility trade-off—is obtained even though tree selection itself is scalar, which is a conservative test of the front-keeping contribution.

Empirical check. We directly tested whether this separation costs anything by adding a ParetoPUCT-style selection variant that, at every node, restricts the candidates to the non-dominated children (on the mean multi-objective vectors) before applying the PUCT score (`selection_mode="pareto"` in `p4_mcts_pareto.py`). At equal budget (700 iterations, 5 seeds) the merged fronts are indistinguishable under a common normalisation (hypervolume 15.56 for the scalar-proxy rule vs 15.49 for the Pareto-only rule, $\Delta \approx 0.5\%$). This confirms that the scalar proxy is not a bottleneck: the returned front is attributable to the global non-dominated pool and the policy-guided exploration, both shared by the two variants.